

UNIwersYTET RZESZOWSKI
WYDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH
INSTYTUT INFORMATYKI



Diana Vaida
134981

Informatyka

System zarządzania biblioteką – projekt i implementacja w C#

Praca projektowa

Prowadzący
mgr inż. Ewa Żesławska

Rzeszów 2026

Spis treści

1. Streszczenie w języku polskim i angielskim	6
2. Wprowadzenie i analiza problemu	7
3. Wymagania systemowe	8
4. Architektura i struktura aplikacji	9
5. Technologie wykorzystane w projekcie	11
6. Projekt bazy danych	12
7. Implementacja aplikacji	14
8. Harmonogram realizacji projektu	18
9. Instrukcja uruchomienia projektu	19
10. Prezentacja warstwy użytkowej projektu	22
11. Podsumowanie	30
Bibliografia	31
Spis rysunków	32
Spis tabel	33
Spis listingów	34
Oświadczenie studenta o samodzielności pracy	35

1. Streszczenie w języku polskim i angielskim

W danym projekcie zrealizowany został system zarządzania biblioteką. Wszystkie dane są pobierane z bazy, co umożliwia mobilność programu dla użytkowników. System pozwala na łatwe założenie konta czytelnika. Wyszukiwarka umożliwia znalezienie książek po tytule, autorze i kategorii. Wypożyczeń można dokonywać z poziomu konta czytelnika, gdzie wyświetlane są również ewentualne zaległości. Zwrotu książek dokonuje konto bibliotekarza. Bibliotekarz ma możliwość dodawania, edytowania i usuwania książek z bazy. Może także przeglądać wypożyczenia czytelników oraz oznaczać książki jako zwrócone. System pozwala bibliotekarzowi na eksportowanie i importowanie danych (użytkownicy, wypożyczenia, książki).

In a given project, a library management system is realized. All data is extracted from the database, which allows mobility of the program for users. Also, it allows easy creation of a reader's account. Search allows finding books by title, author, and category. Borrowing can be done via the reader's account, and backlogs are displayed. Returning books is done through the librarian's account. The librarian can add, edit, and delete books. Also, they can see the readers' loans and mark a book as returned. The librarian can export and import data (users, loans, books).

2. Wprowadzenie i analiza problemu

Celem tego projektu jest efektywne zarządzanie danymi biblioteki. Aplikacja umożliwia łatwe przeglądanie, edytowanie oraz dodawanie danych do bazy. Dzięki intuicyjnemu interfejsowi użytkownik nie musi posiadać zaawansowanej wiedzy na temat systemów zarządzania bazami danych. Obecnie baza działa lokalnie, jednak architektura pozwala na wdrożenie jej na serwer online.

Wymagania funkcjonalne

- Umożliwia zakładanie kont czytelników i logowanie do systemu (z uwzględnieniem ról).
- Pozwala na wyszukiwanie książek według tytułu, autora i kategorii.
- Pozwala wyszukiwać użytkowników według imienia, nazwiska, nazwy użytkownika i roli.
- Użytkownicy mogą wypożyczać książki, a system śledzi zaległości i ewentualne kary.
- Bibliotekarz ma możliwość dodawania, edytowania i usuwania książek.
- System umożliwia zwrot książek poprzez konto bibliotekarza.
- Eksportowanie i importowanie danych do bazy biblioteki poprzez konto bibliotekarza.
- Zmiana hasła.

Wymagania niefunkcjonalne

- System powinien zapewniać wysoką wydajność w przetwarzaniu zapytań do bazy danych.
- Interfejs użytkownika (GUI) powinien być intuicyjny, estetyczny i łatwy w obsłudze.
- System powinien być skalowalny i bezpieczny, aby obsłużyć rosnącą liczbę użytkowników.

Analiza rozwiązań o podobnym przeznaczeniu W celu zdefiniowania wymagań funkcjonalnych systemu, przeanalizowano trzy popularne rozwiązania biblioteczne:

- Koha (Open-Source) [5] – globalny system webowy. Oferuje pełną obsługę cyklu wypożyczeń, automatyczną kontrolę stanów magazynowych oraz automatyczne naliczanie kar za przetrzymanie książek. Opiera się na skomplikowanych formatach bibliotecznych (MARC21).
- MAK+ (MakPlus) [4] – polski system chmurowy Instytutu Książki. Wdraża system ról (bibliotekarz/czytelnik) oraz automatyczne blokady kont dla dłużników. Umożliwia pobieranie danych o książkach bezpośrednio z bazy centralnej po kodzie ISBN.
- MOL NET+ [7] – komercyjny system chmurowy, standard w polskich szkołach. Posiada intuicyjny panel administratora, automatyczną integrację z e-dziennikami oraz moduły do zarządzania kopiami zapasowymi.

3. Wymagania systemowe

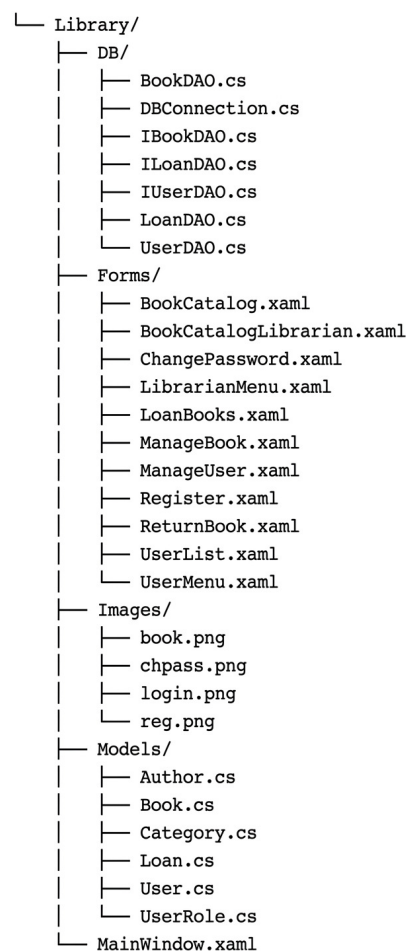
Aplikacja została zaprojektowana na platformę .NET. Minimalne wymagania do uruchomienia to:

- Procesor: Intel i3 / AMD Ryzen 3.
- Pamięć RAM: Minimum 4 GB.
- System operacyjny: Windows 10/11.
- .NET Desktop Runtime (kompatybilny z wersją użytą w projekcie).

Do otwarcia projektu i jego dalszego rozwoju zalecane jest pobranie środowiska Microsoft Visual Studio (18.5.2) ze strony <https://visualstudio.microsoft.com/downloads/>. Dodatkowe narzędzie to SQL Server Management Studio 22 (22.5.2) można pobrać na stronie <https://learn.microsoft.com/en-us/ssms/install/install>.

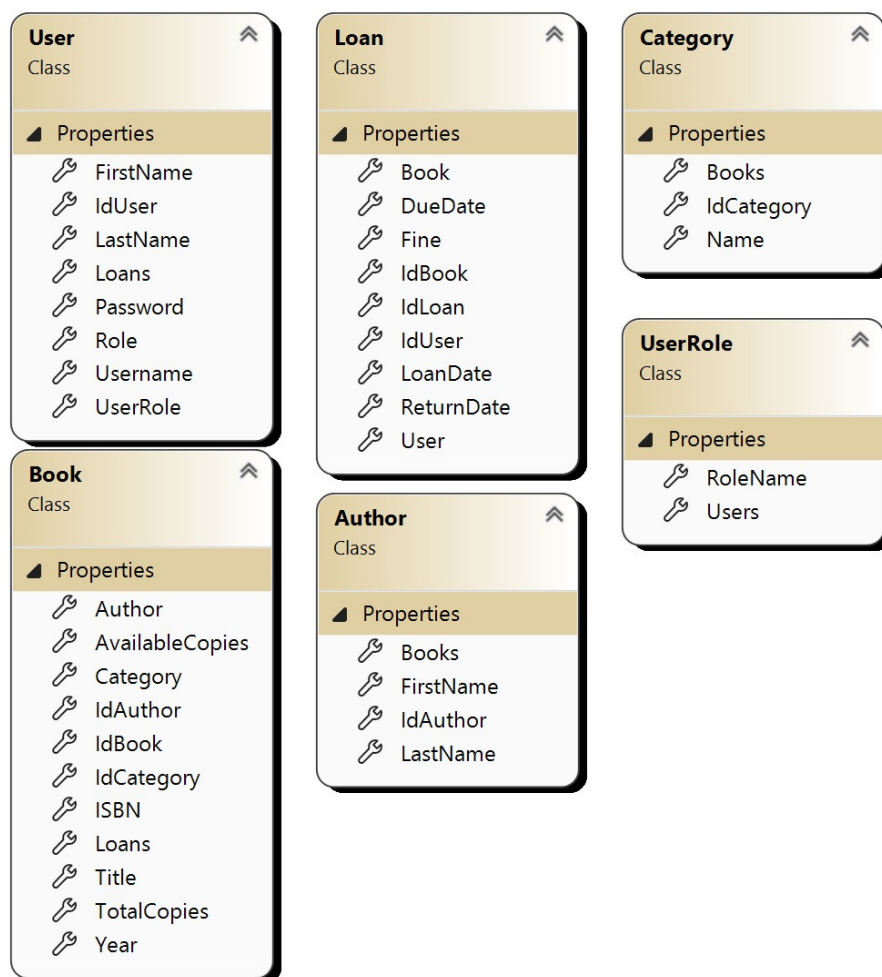
4. Architektura i struktura aplikacji

Architektura aplikacji została podzielona na warstwy zgodnie z dobrymi praktykami programowania w C# (WPF). Wyróżniamy podział na modele, bazę danych, interfejs graficzny i logikę biznesową.



Rys. 4.1. Struktura folderów.

W folderze `Models` zdefiniowano obiekty (klasy encyjne) reprezentujące tabele z bazy danych. Są to: `User`, `Loan`, `Category`, `Book`, `Author` oraz `UserRole`. Klasy te odzwierciedlają strukturę relacyjną; zawierają właściwości odpowiadające kolumnom w tabelach (np. `FirstName`, `LastName`, `ISBN`) oraz właściwości nawigacyjne używane przez Entity Framework do obsługi relacji między obiektami (np. kolekcja `Loans` w klasie `User`).



Rys. 4.2. Diagram klas - modele bazodanowe.

W folderze DB znajduje się logika dostępu do danych, a Forms zawiera definicje widoków interfejsu. Mechanizmy walidacji wprowadzanych danych i obsługa wyjątków zostały zaimplementowane tak, aby system był stabilny i informował użytkownika o błędach.

5. Technologie wykorzystane w projekcie

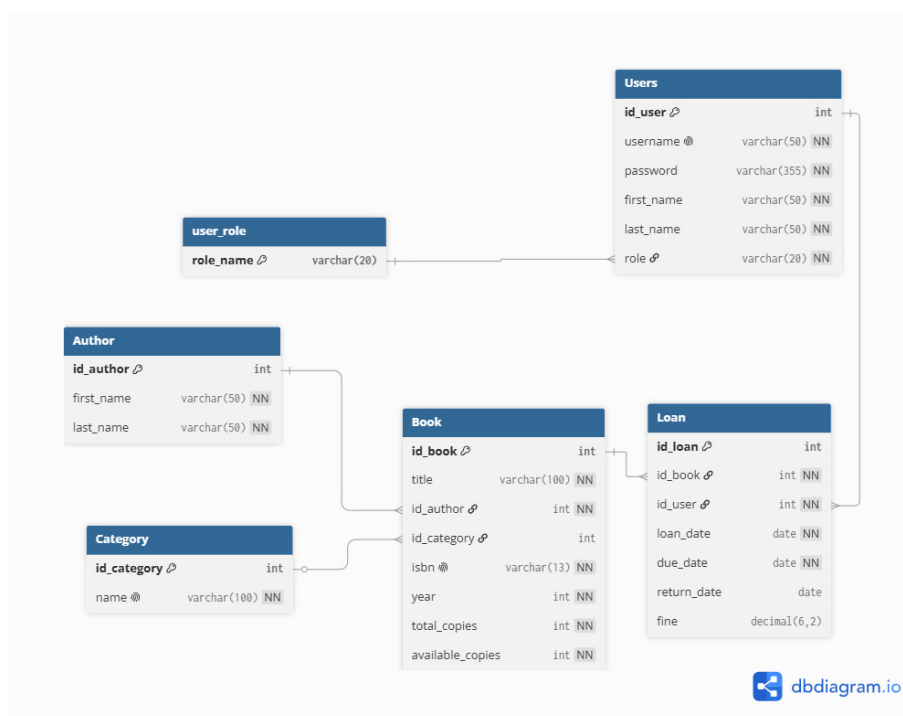
- **C# (.NET)** [1]: Główny język programowania aplikacji. Został wybrany ze względu na jego silne typowanie, pełne wsparcie dla paradygmatu programowania obiektowego oraz bogaty zbiór wbudowanych bibliotek w środowisku .NET, które znacznie przyspieszają proces tworzenia zaawansowanych i stabilnych aplikacji desktopowych.
- **WPF (Windows Presentation Foundation)** [8]: Technologia użyta do zaprojektowania interfejsu użytkownika. Zdecydowano się na WPF ze względu na elastyczność i wyraźne oddzielenie warstwy wizualnej (zapisanej w deklaratywnym języku XAML) od logiki biznesowej w C#. Dodatkowo WPF posiada zaawansowane mechanizmy wiązania danych (Data Binding), co idealnie współgra z dynamicznym wyświetlaniem informacji z bazy.
- **Microsoft SQL Server** [6]: Relacyjny system zarządzania bazą danych (RDBMS). Wykorzystano go do trwałego i bezpiecznego przechowywania wszelkich informacji systemowych (użytkownicy, książki, operacje wypożyczeń). Jego wybór podyktowany był bardzo dobrą, natywną integracją z platformą .NET, wysoką wydajnością i świetnymi mechanizmami zabezpieczającymi integralność relacyjną tabel.
- **Entity Framework Core** [3]: Narzędzie typu ORM (Object-Relational Mapping). Użyto go, aby móc operować na bazie danych bezpośrednio za pomocą bezpiecznie typowanych obiektów w języku C#, zamiast pisać skomplikowane zapytania SQL w tekście. Usprawniło to proces mapowania struktury bazy na klasy modelu (tzw. encje) oraz przyspieszyło tworzenie operacji CRUD, dodatkowo eliminując ryzyko błędów pokroju SQL Injection.

6. Projekt bazy danych

Baza danych została zaprojektowana z zachowaniem zasad normalizacji i integralności danych, wykorzystując możliwości silnika SQL. Składa się z następujących tabel:

- **user_role** – słownik ról systemowych (zawiera wartości 'reader' i 'librarian'). Posiada jedno pole `role_name` pełniące funkcję klucza głównego.
- **Users** – przechowuje informacje o użytkownikach (bibliotekarzach i czytelnikach). Zawiera unikalną nazwę użytkownika (`username`), zahashowane hasło (`password`), imię, nazwisko oraz przypisaną rolę, która jest kluczem obcym powiązanym z tabelą `user_role`.
- **Author** – słownik autorów książek, zawierający imię i nazwisko autora wraz z automatycznie generowanym kluczem `id_author` (IDENTITY).
- **Category** – słownik kategorii literackich, dbający o to, by nazwa każdej kategorii była unikalna (UNIQUE).
- **Book** – główna tabela przechowująca zasoby biblioteki. Posiada klucze obce do tabel `Author` oraz `Category`. Gwarantuje unikalność 13-znakowego numeru `isbn` oraz dba o integralność liczbową za pomocą ograniczeń CHECK (całkowita i dostępna liczba kopii nie może być mniejsza od zera, a liczba dostępna nie może przekroczyć całkowitej).
- **Loan** – tabela asocjacyjna realizująca relację wypożyczeń między użytkownikami a książkami. Zapisuje datę wypożyczenia (`loan_date`), przewidywany termin zwrotu (`due_date`) oraz rzeczywistą datę zwrotu (`return_date`). Zawiera także kwotę ewentualnej kary (`fine`). Zastosowano tutaj mechanizmy kaskadowego usuwania (ON DELETE CASCADE).

Aby zapewnić ścisłą kontrolę nad wprowadzanymi danymi, szeroko wykorzystano więzy spójności typu CHECK (np. sprawdzenie poprawności długości numeru ISBN oraz relacji czasowych między datami wypożyczenia a terminem zwrotu) oraz restrykcje kluczy obcych.

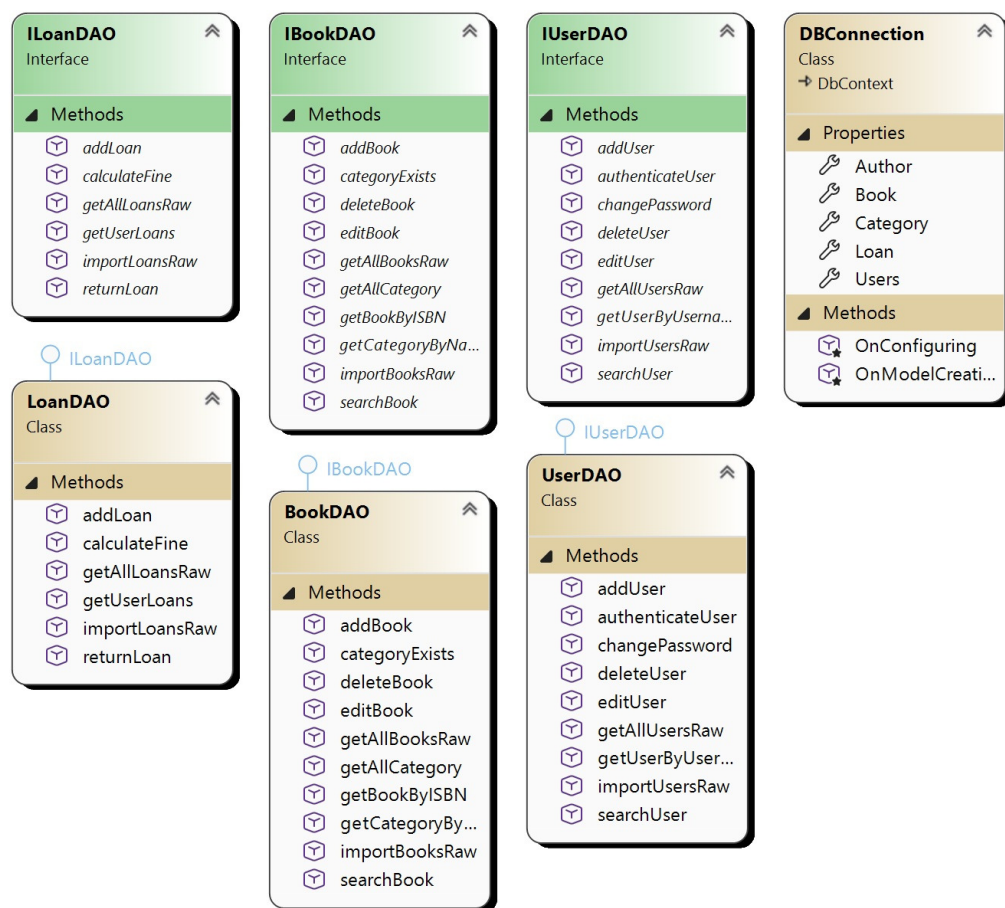


Rys. 6.1. Diagram ERD projektowanej bazy danych. [2]

7. Implementacja aplikacji

Najważniejsze moduły systemu to uwierzytelnianie użytkowników, moduł zarządzania katalogiem książek oraz obsługa procesu wypożyczeń i zwrotów. Interakcje z bazą danych operacje CRUD zostały zrealizowane za pomocą wzorca DAO oraz biblioteki **Entity Framework Core**. Kod odpowiedzialny za komunikację z bazą zgromadzono w osobnym folderze DB.

Struktura logiki dostępu do danych opiera się na interfejsach: `ILoanDAO`, `IBookDAO` oraz `IUserDAO`, które definiują kontrakty (metody) niezbędne do obsługi poszczególnych encji (np. `addBook`, `searchBook`, `authenticateUser`, `returnLoan`). Klasy `LoanDAO`, `BookDAO` i `UserDAO` implementują te interfejsy. Całość komunikacji z silnikiem SQL koordynuje klasa `DBConnection`, dziedzicząca po `DbContext` z Entity Framework, która przechowuje zbiory danych (`DbSet`) dla każdej encji.



Rys. 7.1. Diagram klas - warstwa dostępu do danych (DAO).

Do głównych komponentów warstwy dostępu do danych należą:

- **BookDAO** – obsługuje wyszukiwanie książek, dodawanie, edycję i usuwanie zasobów. Dbą również o zarządzanie relacjami. Przy dodawaniu lub edytowaniu książki system automatycznie sprawdza, czy dany autor lub kategoria już istnieją, a jeśli nie, tworzy ich nowe wpisy w bazie.
- **UserDAO** – realizuje operacje na kontach użytkowników. Umożliwia bezpieczne logowanie i autoryzację z wykorzystaniem hashowania haseł (dzięki mechanizmom `PasswordHasher`). Ponadto

pozwała na rejestrację, edycję, usuwanie profilu oraz zmianę hasła. Zadbano również o to, aby usunięcie użytkownika automatycznie przywracało status dostępności dla niezwróconych przez niego książek.

- **LoanDAO** – zarządza logiką biznesową wypożyczeń. Zapewnia rezerwację książki (zmniejszenie licznika dostępnych egzemplarzy) i wyznacza termin zwrotu na 14 dni od momentu wypożyczenia. Zwrot książki podnosi licznik z powrotem. Istotnym elementem jest funkcja naliczająca kary (`calculateFine`), która automatycznie generuje opłatę karną w wysokości 0.50 jednostek za każdy dzień opóźnienia.

W systemie zaimplementowano również ważny moduł odpowiedzialny za masowy eksport i import danych (kopie zapasowe). Każda klasa DAO dostarcza metody `getAll...Raw()`, które pobierają ze struktury pełne kolekcje rekordów do wyeksportowania do pliku. Z kolei metody `import...Raw()` realizują bezpieczne wprowadzanie danych z powrotem do systemu – usuwają one najpierw istniejące zbiory z bazy, a następnie wprowadzają listę dostarczonych encji, zapewniając synchronizację stanu bazy danych z zaimportowanym plikiem.

Każda operacja wykonywana jest w bezpiecznym bloku klasy zarządzającej połączeniem (`DBConnection`), gwarantując poprawne otwarcie i zamknięcie sesji bazy danych.

Listing 7.1. Klasa łącząca się z bazą danych

```

1 using Library.Models;
2 using Microsoft.EntityFrameworkCore;
3 using System;
4 using System.Collections.Generic;
5 using System.Text;
6
7 namespace Library.DB
8 {
9     class DBConnection : DbContext
10    {
11        public DbSet<User> Users { get; set; }
12
13        public DbSet<Author> Author { get; set; }
14
15        public DbSet<Category> Category { get; set; }
16
17        public DbSet<Book> Book { get; set; }
18
19        public DbSet<Loan> Loan { get; set; }
20        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
21        {
22            optionsBuilder.UseSqlServer(
23                "Server=localhost\\SQLEXPRESS;Database=LibraryDB;Trusted_Connection=True
;TrustServerCertificate=True"
24            );
25        }
26
27        protected override void OnModelCreating(ModelBuilder modelBuilder)
28        {
29            modelBuilder.Entity<Book>(entity =>
30            {
31                entity.ToTable("Book");
32                entity.HasKey(x => x.IdBook);

```

```

33         entity.Property(x => x.IdBook).HasColumnName("id_book");
34         entity.Property(x => x.IdAuthor).HasColumnName("id_author");
35         entity.Property(x => x.IdCategory).HasColumnName("id_category");
36         entity.Property(x => x.Year).HasColumnName("year");
37         entity.Property(x => x.ISBN).HasColumnName("isbn");
38         entity.Property(x => x.TotalCopies).HasColumnName("total_copies");
39         entity.Property(x => x.AvailableCopies).HasColumnName("available_copies"
);
40
41         entity.HasOne(d => d.Author)
42             .WithMany(p => p.Books)
43             .HasForeignKey(d => d.IdAuthor);
44
45         entity.HasOne(d => d.Category)
46             .WithMany(p => p.Books)
47             .HasForeignKey(d => d.IdCategory);
48     });
49
50     modelBuilder.Entity<Loan>(entity =>
51     {
52         entity.ToTable("Loan");
53         entity.HasKey(x => x.IdLoan);
54         entity.Property(x => x.IdLoan).HasColumnName("id_loan");
55         entity.Property(x => x.IdBook).HasColumnName("id_book");
56         entity.Property(x => x.IdUser).HasColumnName("id_user");
57         entity.Property(x => x.LoanDate).HasColumnName("loan_date");
58         entity.Property(x => x.DueDate).HasColumnName("due_date");
59         entity.Property(x => x.ReturnDate).HasColumnName("return_date");
60
61         entity.Property(x => x.Fine)
62             .HasColumnName("fine")
63             .HasColumnType("decimal(6,2)")
64             .HasDefaultValue(0.00m);
65
66         entity.HasOne(d => d.Book)
67             .WithMany(p => p.Loans)
68             .HasForeignKey(d => d.IdBook)
69             .OnDelete(DeleteBehavior.Restrict);
70
71         entity.HasOne(d => d.User)
72             .WithMany(p => p.Loans)
73             .HasForeignKey(d => d.IdUser)
74             .OnDelete(DeleteBehavior.Restrict);
75     });
76
77     modelBuilder.Entity<Author>(entity =>
78     {
79         entity.ToTable("Author");
80         entity.HasKey(x => x.IdAuthor);
81         entity.Property(x => x.IdAuthor).HasColumnName("id_author");
82         entity.Property(x => x.FirstName).HasColumnName("first_name");
83         entity.Property(x => x.LastName).HasColumnName("last_name");
84     });
85
86     modelBuilder.Entity<Category>(entity =>
87     {
88         entity.ToTable("Category");

```

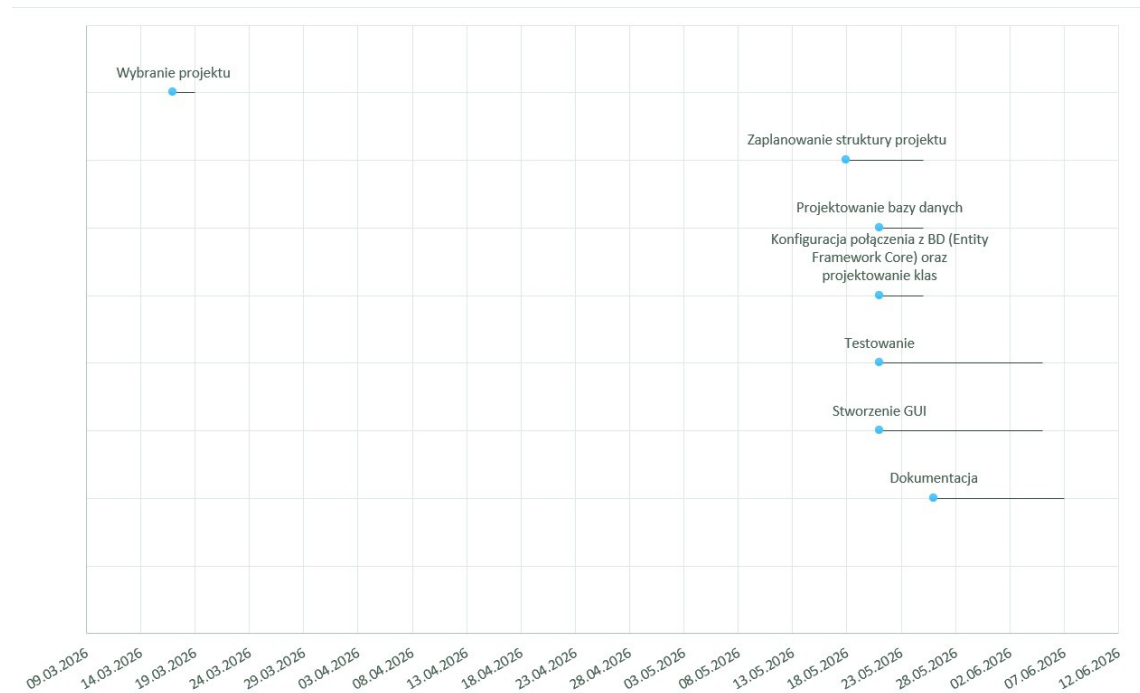
```
89         entity.HasKey(x => x.IdCategory);
90         entity.Property(x => x.IdCategory).HasColumnName("id_category");
91     });
92
93     modelBuilder.Entity<UserRole>(entity =>
94     {
95         entity.ToTable("user_role");
96         entity.HasKey(x => x.RoleName);
97         entity.Property(x => x.RoleName).HasColumnName("role_name");
98     });
99
100    modelBuilder.Entity<User>(entity =>
101    {
102        entity.ToTable("Users");
103        entity.HasKey(x => x.IdUser);
104        entity.Property(x => x.IdUser).HasColumnName("id_user");
105        entity.Property(x => x.FirstName).HasColumnName("first_name");
106        entity.Property(x => x.LastName).HasColumnName("last_name");
107
108        entity.HasOne(x => x.UserRole)
109            .WithMany(x => x.Users)
110            .HasForeignKey(x => x.Role);
111    });
112
113    }
114
115    }
116 }
```

Na wyższych warstwach aplikacji mechanizmy autoryzacji ograniczają widoczność poszczególnych widoków WPF w zależności od roli zalogowanej osoby.

8. Harmonogram realizacji projektu

Kod źródłowy projektu, historia zmian oraz zarządzanie wersjami odbywało się z wykorzystaniem systemu kontroli wersji Git. Repozytorium projektu dostępne jest pod adresem: <https://github.com/Adoreconfer/Projekt>.

Harmonogram prac nad aplikacją został podzielony na kilka kluczowych etapów, których realizacja postępowała w określonych ramach czasowych

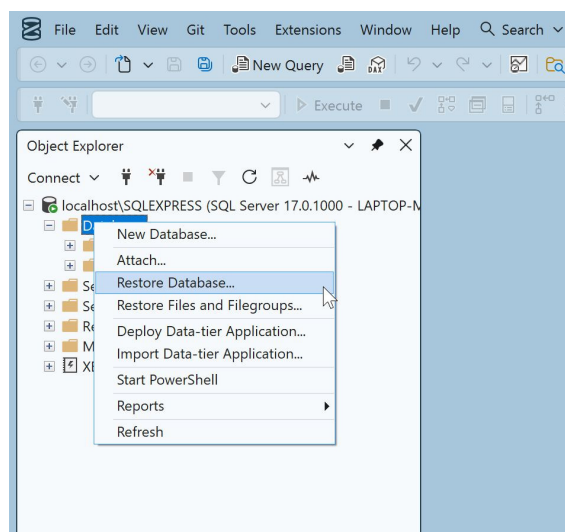


Rys. 8.1. Diagram Gantta przedstawiający etapy powstawania aplikacji.

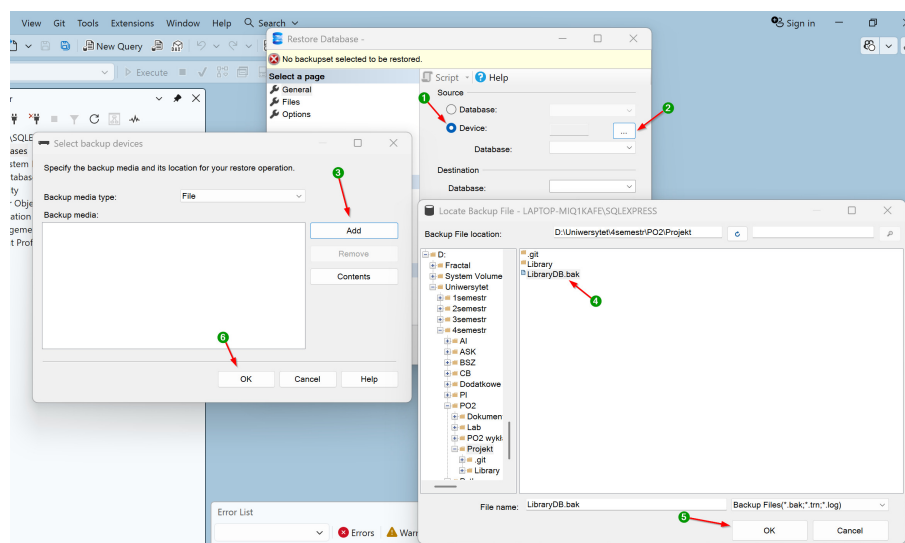
9. Instrukcja uruchomienia projektu

Aby poprawnie uruchomić projekt na urządzeniu końcowym, wykonaj następujące kroki:

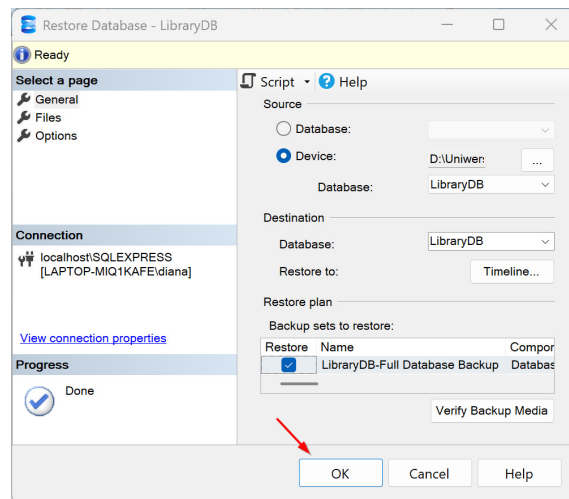
1. Upewnij się, że posiadasz zainstalowane środowisko **Microsoft Visual Studio** oraz serwer lokalny **SQL Server**.
2. Utwórz bazę danych w SQL Server i odtwórz ją korzystając z dostarczonego backup `LibraryDB.bak`.



Rys. 9.1. Przywracanie bazy danych z pliku backupu (cz. 1).



Rys. 9.2. Przywracanie bazy danych z pliku backupu (cz. 2).

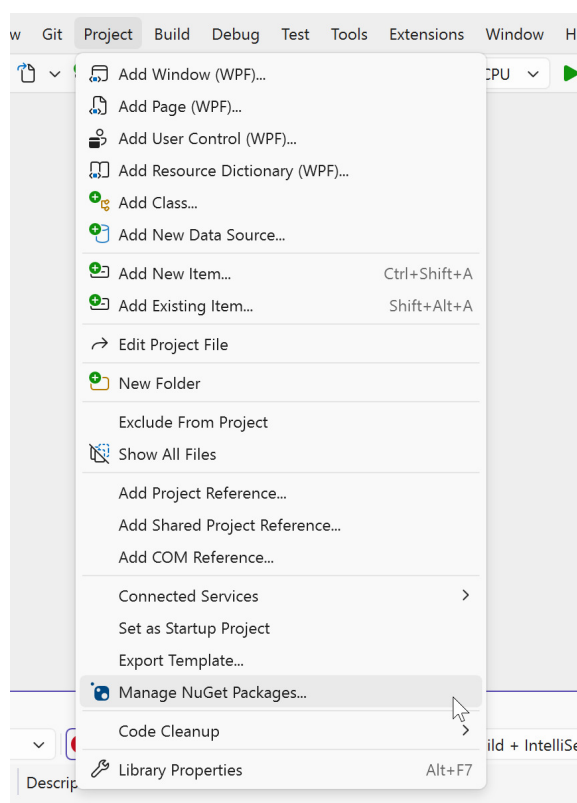


Rys. 9.3. Przywracanie bazy danych z pliku backupu (cz. 3).

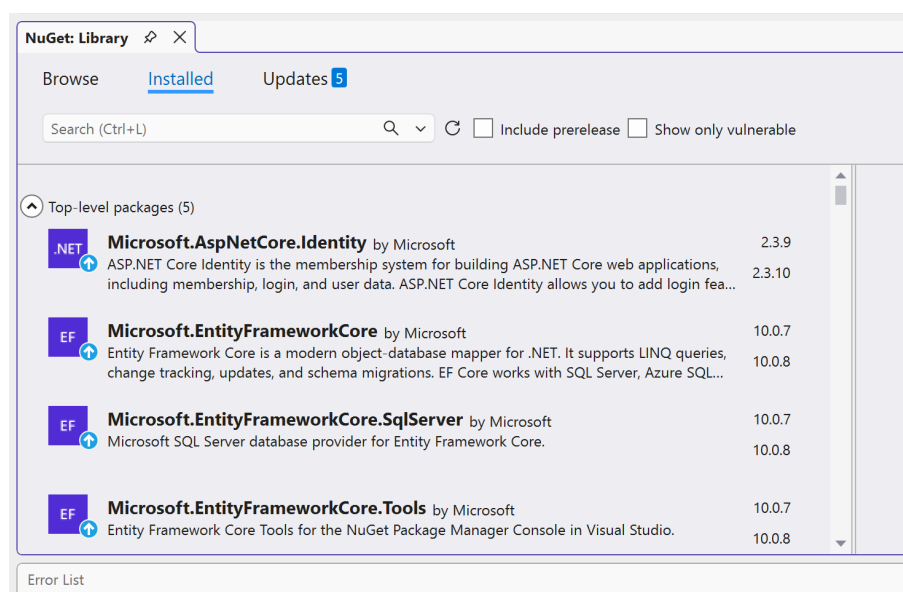
3. Otwórz rozwiązanie projektu C# `Library.sln` w Visual Studio.

4. Upewnij się, że masz zainstalowane i przywrócone wszystkie niezbędne pakiety NuGet wymagane do działania projektu. Są to następujące pakiety:

- `Microsoft.AspNetCore.Identity`
- `Microsoft.EntityFrameworkCore`
- `Microsoft.EntityFrameworkCore.SqlServer`
- `Microsoft.EntityFrameworkCore.Tools`
- `Microsoft.Extensions.Identity.Core`



Rys. 9.4. Zarządzanie pakietami NuGet - pobrane biblioteki (cz. 1).



Rys. 9.5. Zarządzanie pakietami NuGet - pobrane biblioteki (cz. 2).

5. Zbuduj projekt i uruchom (Start / F5).

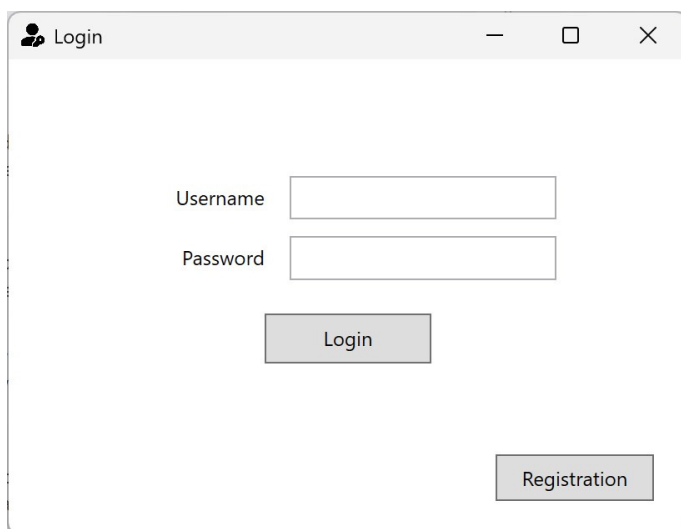
Dane testowe: Aby przetestować działanie poszczególnych ról w systemie, po zaimportowaniu bazy danych można zalogować się używając poniższych poświadczeń:

- Bibliotekarz: Login: admin, Hasło: admin1234
- Czytelnik: Login: anna, Hasło: anna1234

10. Prezentacja warstwy użytkowej projektu

Graficzny interfejs użytkownika (GUI) został zaprojektowany w technologii WPF z naciskiem na przejrzystość, ergonomię i intuicyjną obsługę. Przepływ pracy (User Flow) w aplikacji rozpoczyna się od autoryzacji, a następnie kieruje użytkownika do dedykowanego panelu w zależności od nadanych mu uprawnień.

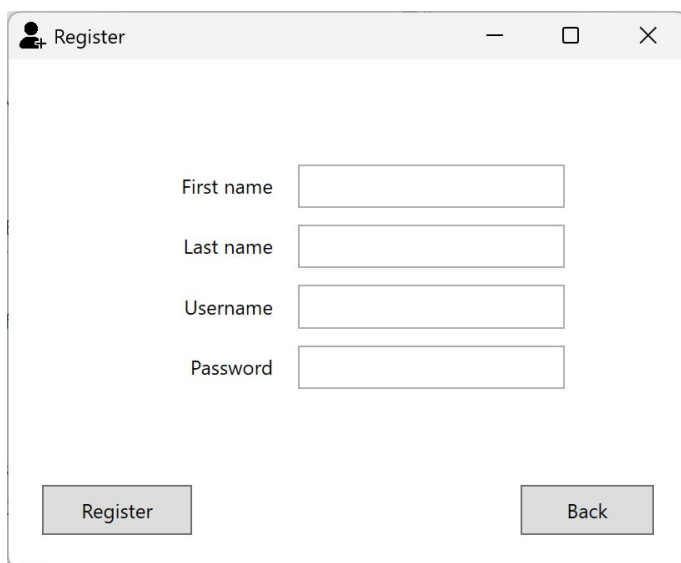
Pierwszym oknem dialogowym, z którym styka się użytkownik, jest ekran logowania. Pozwala on na wprowadzenie nazwy użytkownika i hasła, weryfikowanych następnie w bazie danych. Okno zawiera również przycisk pozwalający na przejście do formularza utworzenia nowego konta.



The screenshot shows a standard Windows-style window titled "Login". Inside, there are two text input fields, one labeled "Username" and one labeled "Password". Below the "Password" field is a rectangular button labeled "Login". In the bottom right corner of the window, there is another rectangular button labeled "Registration".

Rys. 10.1. Ekran logowania do systemu.

W przypadku wyboru opcji założenia konta, aplikacja przenosi do panelu rejestracji. Wymaga on uzupełnienia podstawowych danych osobowych oraz ustalenia własnych danych dostępowych do systemu.

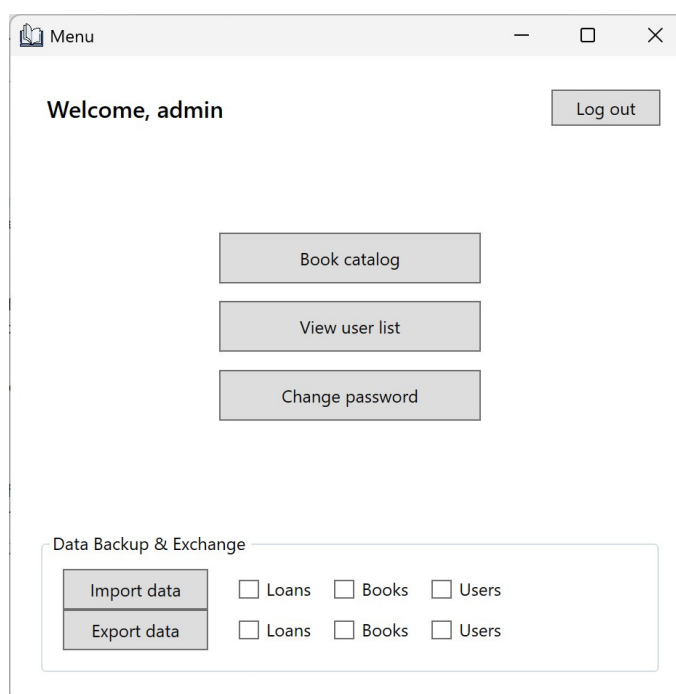


The screenshot shows a standard Windows-style window titled "Register". Inside, there are four text input fields arranged vertically, labeled "First name", "Last name", "Username", and "Password". At the bottom left of the window is a rectangular button labeled "Register". At the bottom right is another rectangular button labeled "Back".

Rys. 10.2. Ekran rejestracji nowego użytkownika.

Po pomyślnym zalogowaniu z uprawnieniami administracyjnymi (jako bibliotekarz), użytkownikowi ukazuje się główne menu nawigacyjne. Udostępnia ono scentralizowany dostęp do kluczowych procesów systemu, pozwalając na przejście do zarządzania zasobami biblioteki (*Book catalog*), kontrolowania użytkowników (*View user list*) oraz zmiany hasła.

Istotnym elementem tego widoku jest moduł *Data Backup & Exchange* umieszczony w dolnej części interfejsu. Umożliwia on administratorowi wybiórcze zarządzanie kopiami zapasowymi poszczególnych danych (za pomocą pól wyboru dla tabel: Loans, Books, Users), a następnie wyeksportowanie ich do zewnętrznych plików lub zaimportowanie z powrotem w celu odtworzenia stanu biblioteki.



Rys. 10.3. Główne menu administratora z panelem zarządzania kopiami zapasowymi.

Kluczowym modułem dla bibliotekarza jest zarządzanie zbiorami, dostępne pod opcją *Book catalog*. Wyświetla ono czytelną tabelę ze wszystkimi pozycjami w bibliotece. Umożliwia zaawansowane wyszukiwanie książek (po frazie zawartej we wszystkich polach, konkretnym tytule lub nazwisku autora) oraz filtrowanie wyników na podstawie wybranej kategorii. Z tego poziomu administrator ma również kontrolę nad zasobami, korzystając z umieszczonych na dole przycisków służących do dodawania, edycji lub usuwania zaznaczonej pozycji.

The 'Catalog' window displays a search interface and a table of books. At the top, there is a search bar with a 'Search' button and a 'Category' dropdown menu set to 'All'. Below the search bar are three radio buttons: 'All' (selected), 'Title', and 'Author'. The table lists books with columns for Title, Author, ISBN, Year, Category, Total, and Available. Below the table is a large empty rectangular area. At the bottom of the window are four buttons: 'Back', 'Add book', 'Edit book', and 'Delete book'.

Title	Author	ISBN	Year	Category	Total	Available
Quo Vadis	Henryk Sienkiewicz	9788373271890	1896	Powiesc historyczna	10	10
Pan Tadeusz	Adam Mickiewicz	9788307032912	1834	Poemat	8	8
Solaris	Stanislaw Lem	9788373191723	1961	Science Fiction	12	12
Bieguni	Olga Tokarczuk	9788308063007	2007	Literatura współczesna	6	6
Wiedźmin: Ostatni	Andrzej Sapkowski	9788375780635	1993	Fantasy	15	15

Rys. 10.4. Katalog książek z wyszukiwarką i opcjami zarządzania (widok bibliotekarza).

Kliknięcie przycisku dodawania nowej książki otwiera dedykowany formularz. Bibliotekarz musi wprowadzić niezbędne informacje ewidencyjne, takie jak: tytuł, autor, kategoria (z możliwością wyboru istniejącej z rozwijanej listy lub wpisania nowej poprzez tryb *New*), numer ISBN, rok publikacji oraz całkowitą liczbę wprowadzanych na stan egzemplarzy.

The 'Add book' window contains a form for adding a new book. It has six input fields: 'Title', 'Author', 'Category' (with a 'New' dropdown button), 'ISBN', 'Publication year', and 'Total copies'. At the bottom of the form are two buttons: 'Add book' and 'Back'.

Rys. 10.5. Formularz dodawania nowej książki do bazy.

Modyfikacja metadanych książki odbywa się za pomocą bliźniaczego formularza wywoływanego opcją *Edit book*. Przy uruchomieniu tego okna, wszystkie pola tekstowe są automatycznie uzupełniane danymi wybranej uprzednio z katalogu pozycji. Umożliwia to szybką korektę, na przykład poprawę literówek, przypisanie nowej kategorii, czy aktualizację liczby fizycznych kopii w magazynie.

The 'Edit book' window contains the following fields and controls:

- Title: Quo Vadis
- Author: Henryk Sienkiewicz
- Category: Powiesc historyczna (dropdown menu)
- ISBN: 9788373271890
- Publication year: 1896
- Total copies: 10
- Buttons: Edit book, Back

Rys. 10.6. Formularz edycji danych istniejącej książki.

Bibliotekarz ma dostęp do listy wszystkich zarejestrowanych użytkowników (*View user list*). Widok ten udostępnia tabelę z danymi kont (Id, nazwisko, imię, nazwa użytkownika, rola) oraz wyszukiwarkę pozwalającą filtrować użytkowników po imieniu, nazwisku lub loginie, a także według roli (*Reader/Librarian*). Dostępne przyciski pozwalają na podgląd szczegółów, edycję, dodanie nowego konta oraz usunięcie wybranego użytkownika.

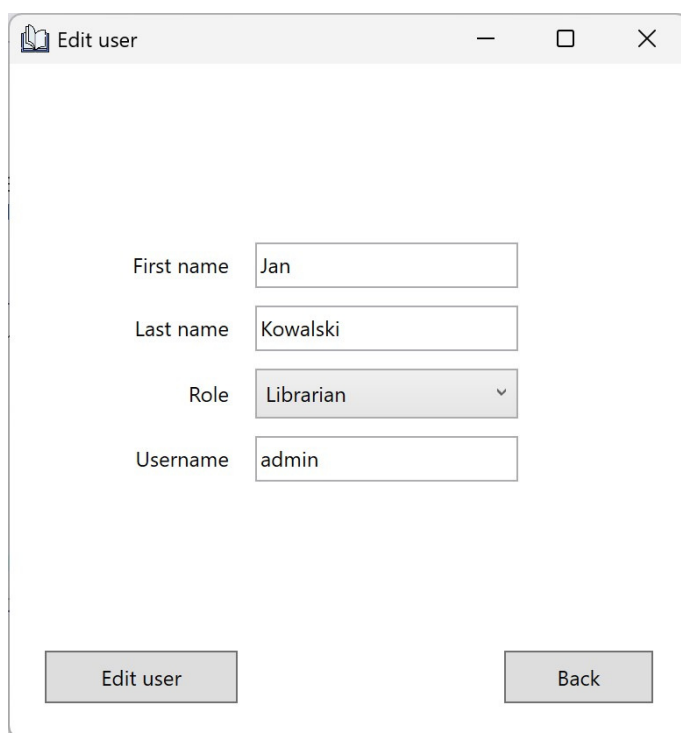
The 'User List' window includes a search bar, a 'Search' button, and a 'Role:' dropdown menu. Below these are radio buttons for filtering: All (selected), First name, Last name, and Username. The table below shows the user data:

Id	Last name	First name	Username	Role
1	Kowalski	Jan	admin	librarian
2	Nowak	Anna	anna	reader

At the bottom of the window are buttons for: Back, More info, Edit user, Add user, and Delete.

Rys. 10.7. Lista użytkowników z opcjami zarządzania kontami.

Formularz edycji użytkownika, dostępny po wybraniu konta z listy i kliknięciu *Edit user*, wczytuje automatycznie aktualne dane wybranej osoby. Bibliotekarz może zmodyfikować imię, nazwisko, nazwę użytkownika oraz zmienić przypisaną rolę za pomocą rozwijanej listy.



The image shows a web browser window with the title "Edit user". Inside the window is a form with the following fields:

- First name:** A text input field containing the text "Jan".
- Last name:** A text input field containing the text "Kowalski".
- Role:** A dropdown menu with "Librarian" selected and a downward arrow.
- Username:** A text input field containing the text "admin".

At the bottom of the form, there are two buttons:

- Edit user:** A button to save the changes.
- Back:** A button to return to the previous screen.

Rys. 10.8. Formularz edycji danych użytkownika.

Opcja *Add user* otwiera formularz tworzenia nowego konta z poziomu panelu administratora. W odróżnieniu od rejestracji dostępnej dla niezalogowanych, ten widok pozwala bibliotekarzowi bezpośrednio przypisać rolę tworzonemu kontu (Reader lub Librarian). Formularz wymaga uzupełnienia imienia, nazwiska, nazwy użytkownika, hasła i potwierdzenia hasła.

First name

Last name

Role Reader ▾

Username

Password

Confirm password

Rys. 10.9. Formularz dodawania nowego użytkownika przez bibliotekarza.

Obsługa zwrotów odbywa się przez dedykowany widok *Return Book*. Bibliotekarz widzi tabelę aktywnych wypożyczeń zawierającą wszystkie istotne informacje: dane książki (tytuł, autor, ISBN), daty (data wypożyczenia oraz termin zwrotu), a po dokonaniu zwrotu – datę oddania i ewentualnie naliczoną kwotę kary za opóźnienie (w PLN). Zaznaczenie rekordu i kliknięcie *Return Book* finalizuje operację w systemie.

Id	Title	Author	ISBN	Loan date	Due date	Return date	Fine
1	Quo Vadi	Henryk Sienki	9788373271890	2026-06-06	2026-06-20		0.00 PLN

Rys. 10.10. Panel obsługi zwrotów wypożyczonych książek.

Każdy zalogowany użytkownik (zarówno czytelnik jak i bibliotekarz) ma możliwość zmiany swojego hasła dostępową poprzez opcję *Change password* w menu głównym. Formularz wymaga podania nowego hasła oraz jego potwierdzenia w celu wyeliminowania pomyłki.

Change Password

New password:

Confirm password:

Change password

Back

Rys. 10.11. Formularz zmiany hasła użytkownika.

Wyszukiwarka na liście użytkowników działa dynamicznie – poniższy przykład ilustruje filtrowanie po imieniu (pole *First name*) z wpisaną frazą „A”. System wyświetla wyłącznie pasujące rekordy, w tym przypadku użytkowniczkę Annę Nowak z rolą *reader*.

User List

A Role:

☐ All ☒ First name ☐ Last name ☐ Username

Id	Last name	First name	Username	Role
2	Nowak	Anna	anna	reader

Rys. 10.12. Przykład działania wyszukiwarki na liście użytkowników.

Format pliku eksportu danych to **JSON** (`LibraryBackup.json`). Plik zawiera ustrukturyzowane dane wszystkich wybranych tabel w postaci zagnieżdżonych obiektów. Każdy rekord użytkownika zawiera m.in. pola: `IdUser`, `Username`, `Password` (w postaci zahashowanej, co gwarantuje bezpieczeństwo danych), `FirstName`, `LastName`, `Role` oraz zagnieżdżony obiekt `UserRole` z listą powiązanych encji. Plik ten może zostać ponownie zaimportowany do systemu poprzez mechanizm *Import data*.

```

1  {
2    "Users": [
3      {
4        "IdUser": 1,
5        "Username": "admin",
6        "Password": "AQAAAAIAAYagAAAAEIt2umYGTSB5i0WF0Gu1rDsE\u002B9",
7        "FirstName": "Jan",
8        "LastName": "Kowalski",
9        "Role": "librarian",
10       "UserRole": {
11         "RoleName": "librarian",
12         "Users": [
13           null
14         ]
15       },
16       "Loans": []
17     },
18     {
19       "IdUser": 2,
20       "Username": "anna",
21       "Password": "AQAAAAIAAYagAAAAEEBgCJI7G31GL\u002B8WXQ82id02VNT",
22       "FirstName": "Anna",

```

Rys. 10.13. Przykładowa zawartość pliku eksportu danych LibraryBackup.json.

11. Podsumowanie

Stworzony system pomyślnie zrealizował wszystkie kluczowe założenia projektowe. Aplikacja ułatwia codzienne procesy biblioteczne zarówno z perspektywy pracownika, jak i użytkownika. Napotkane wyzwania techniczne przy konfiguracji WPF zostały rozwiązane poprzez podział widoków. Potencjalnym kierunkiem na przyszły rozwój projektu jest zaimplementowanie rezerwacji książek przez użytkowników, wdrożenie autoryzacji biometrycznej czy utworzenie powiadomień e-mailowych z przypomnieniami o nadchodzących terminach zwrotu.

Bibliografia

- [1] C# documentation. <https://learn.microsoft.com/en-us/dotnet/csharp/>.
- [2] dbdiagram.io. <https://dbdiagram.io/>.
- [3] Entity framework core documentation. <https://learn.microsoft.com/en-us/ef/core/>.
- [4] Instytut książki - program mak+. <https://www.instytutksiazki.pl/programy/mak/>.
- [5] Koha - official website. <https://koha-community.org/>.
- [6] Sql server technical documentation. <https://learn.microsoft.com/en-us/sql/sql-server/>.
- [7] System biblioteczny mol net+ na vulcan. <https://www.vulcan.edu.pl/szkoly-i-przedszkola/oprogramowanie/system-biblioteczny-mol-net>.
- [8] Windows presentation foundation (wpf) for .net – overview. <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/>.

Spis rysunków

4.1	Struktura folderów.	9
4.2	Diagram klas - modele bazodanowe.	10
6.1	Diagram ERD projektowanej bazy danych. [2]	13
7.1	Diagram klas - warstwa dostępu do danych (DAO).	14
8.1	Diagram Gantta przedstawiający etapy powstawania aplikacji.	18
9.1	Przywracanie bazy danych z pliku backupu (cz. 1).	19
9.2	Przywracanie bazy danych z pliku backupu (cz. 2).	19
9.3	Przywracanie bazy danych z pliku backupu (cz. 3).	20
9.4	Zarządzanie pakietami NuGet - pobrane biblioteki (cz. 1).	21
9.5	Zarządzanie pakietami NuGet - pobrane biblioteki (cz. 2).	21
10.1	Ekran logowania do systemu.	22
10.2	Ekran rejestracji nowego użytkownika.	22
10.3	Główne menu administratora z panelem zarządzania kopiami zapasowymi.	23
10.4	Katalog książek z wyszukiwarką i opcjami zarządzania (widok bibliotekarza).	24
10.5	Formularz dodawania nowej książki do bazy.	24
10.6	Formularz edycji danych istniejącej książki.	25
10.7	Lista użytkowników z opcjami zarządzania kontami.	25
10.8	Formularz edycji danych użytkownika.	26
10.9	Formularz dodawania nowego użytkownika przez bibliotekarza.	27
10.10	Panel obsługi zwrotów wypożyczonych książek.	27
10.11	Formularz zmiany hasła użytkownika.	28
10.12	Przykład działania wyszukiwarki na liście użytkowników.	28
10.13	Przykładowa zawartość pliku eksportu danych LibraryBackup.json.	29

Spis tabel

Spis listingów

7.1	Klasa łącząca się z bazą danych	15
-----	---	----

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

.....Diana Vaida.....
Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

.....Informatyka.....
Nazwa kierunku

.....134981.....
Numer albumu

1. Oświadczam, że moja praca projektowa pt.: System zarządzania biblioteką – projekt i implementacja w C#
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/nie wyrażam zgody** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

(miejscowość, data)

(czytelny podpis/y studenta/ów)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić